Heart Disease MLOps

End-to-End ML Model Development, CI/CD, Containerisation, Kubernetes Deployment, and Production Monitoring

Course	**MLOps (S2-25_AMLCSZG523)**
Assignment	Experiential Learning — End-to-End MLOps Pipeline
Student	Kavya Damera
Student ID	2025cs05037
Programme	M.Tech, BITS Pilani — Work Integrated Learning Programmes
Dataset	UCI Heart Disease (Cleveland subset, 303 records)
Repository	https://github.com/BITS-2025cs05037/heart-disease-mlops
Demo recording	[See submission folder — link in §14]
Date of submission	09 May 2026

All code, manifests, monitoring config, screenshots, and the architecture diagram referenced in this report live in the repository above. The repository checkout used to generate this PDF is the 'main' branch at the time of submission.

1. Introduction

This report describes a complete MLOps implementation built around the UCI Heart Disease dataset (Cleveland subset). The brief is intentionally small — a binary classification problem on roughly 300 patient records — but the engineering work around it is treated as if the model were going into production: every step from data download through model serving is reproducible from a clean checkout, version-controlled, automatically tested, packaged as a hardened container, deployed declaratively on Kubernetes, and instrumented for observability.

Throughout the report each design choice is explained inline with its rationale, and the corresponding location in the repository is named so the marker can map any claim back to source. Screenshots that prove the runtime behaviour are placed under the screenshots/ folder of the submission archive (omitted from the GitHub repo to keep it lean) and the architecture diagram is generated by the Python script reports/architecture_diagram.py.

1.1 Snapshot of the final state

Best model	**logistic_regression**
Test ROC-AUC / Accuracy	0.9621 / 88.52%
Test F1 / Recall	0.8814 / 0.9286
Models trained	3 — Logistic Regression, Random Forest, Gradient Boosting
Unit tests	Full pytest suite executed on every push (lint → test → train → image)
CI duration	~4–6 minutes end-to-end on ubuntu-latest, Python 3.11
Container image	heart-disease-api:1.0.0 (multi-stage, non-root UID 10001, RO root FS)
Kubernetes objects	Namespace, Deployment, Service, Ingress, HPA (2–6 pods), NetworkPolicy
Observability	Prometheus + Grafana (docker-compose), 6-panel dashboard
Container runtime	Podman 5.x + Minikube (driver=podman, rootful mode)

The remainder of the report is organised pragmatically rather than by mark weighting: setup first so the work is reproducible, then the data and modelling story, then the platform layers (architecture, CI/CD, container, Kubernetes, observability), and finally the cross-cutting concerns of security, end-to-end reproducibility proof, and the lessons learnt while building the system.

2. Setup and Install

The project is developed and tested on macOS (Apple Silicon, M-series) with Python 3.11 and Podman Desktop. Linux x86_64 with Python 3.11 also works without modification because every container manifest accepts both Docker and Podman (see the CONTAINER_RUNTIME variable in the Makefile). Windows is supported via WSL2 + Podman / Docker Desktop.

2.1 Prerequisites

- Python 3.11 (3.11.10 used during development).
- Podman 5.x (`brew install podman`) or Docker Desktop.
- Minikube 1.34+ (`brew install minikube`) and kubectl ≥ 1.30.
- Optional: Helm 3.x for chart-based install, jq + curl for API smoke tests.

2.2 Clone and bootstrap

```
git clone https://github.com/BITS-2025cs05037/heart-disease-mlops.git
cd heart-disease-mlops
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
```

2.3 Make targets

All common workflows are wrapped in the Makefile so the same commands work on the developer's laptop and inside CI:

Target	What it does
make install-dev	Install runtime + dev dependencies
make download-data	Fetch UCI dataset (with mirror fallback) → data/processed/
make train	Run preprocessing + GridSearchCV, log to MLflow, persist model.pkl
make test	pytest with coverage; HTML report in coverage/
make lint	ruff + black --check on src/, tests/, scripts/
make api	uvicorn src.api:app --reload (FastAPI on :8000)
make mlflow-ui	MLflow UI on :5000 (file:./mlruns backend)
make docker-build / docker-run	Build and run heart-disease-api:latest
make k8s-deploy	kubectl apply -k k8s/ in 'heart-disease' namespace
make report	Regenerate this PDF (LibreOffice headless backend)

2.4 Repository layout

```
heart-disease-mlops/
├── src/                        data loader, preprocessing, training, FastAPI
service
├── tests/                      pytest suite (data, preprocessing, model I/O,
API)
├── notebooks/01_eda.ipynb      exploratory analysis (executed, outputs
stripped)
├── scripts/download_data.py    UCI fetcher with mirror fallback + cleaning
├── docker/Dockerfile           multi-stage hardened image
```

```
├── docker-compose.yml         API + Prometheus + Grafana for local monitoring
├── k8s/                       Namespace, Deployment, Service, Ingress, HPA,
NetworkPolicy
├── helm/heart-disease/        Helm chart (functionally equivalent to k8s/)
├── monitoring/                Prometheus rules + Grafana provisioning +
dashboard JSON
├── .github/workflows/ci-cd.yml  4-stage pipeline (lint → test → train → image)
├── reports/                   PDF generator + architecture diagram script
├── artifacts/                 outputs (plots, model.pkl, metrics.json,
mlruns/)
├── Makefile                   runtime-agnostic developer workflow
└── requirements*.txt          pinned runtime + dev dependencies
```

Note on ports: the local stack uses 8000 (API), 5000 (MLflow), 9090 (Prometheus), and 3000 (Grafana). On macOS, port 5000 may collide with the AirPlay Receiver system service; if that happens disable it from System Settings → AirDrop & Handoff or override the port via `make mlflow-ui MLFLOW_PORT=5050`.

3. Dataset and Exploratory Data Analysis

The dataset is the Cleveland subset of the UCI Heart Disease repository (https://archive.ics.uci.edu/dataset/45/heart+disease), made up of 303 patient records with 13 input features and a multi-class severity target between 0 (no disease) and 4 (severe). Following standard practice for this dataset the target is collapsed to a binary label (0 vs. ≥1) so the problem becomes a clean binary classification.

3.1 Acquisition pipeline

Data acquisition is in scripts/download_data.py. The script first attempts a direct download from the UCI mirror; if that returns a non-200 status (which happens occasionally during UCI maintenance windows) it falls back to the ucimlrepo Python package. After fetch the script:

- Prepends standard column headers (the UCI raw file is header-less).
- Replaces UCI's '?' missing markers with NaN.
- Coerces all feature columns to numeric dtype.
- Drops rows where the target is missing (treated as unrecoverable).
- Binarises 'num' to 0/1 and writes data/processed/heart_disease.csv.

3.2 Feature inventory

Feature	Type	Notes
age	numeric	Patient age in years
sex	categorical	1 = male, 0 = female
cp	categorical	Chest-pain type (1–4)
trestbps	numeric	Resting blood pressure (mm Hg)
chol	numeric	Serum cholesterol (mg/dl); long-tail outliers
fbs	categorical	Fasting blood sugar > 120 mg/dl
restecg	categorical	Resting ECG result
thalach	numeric	Maximum heart rate achieved
exang	categorical	Exercise-induced angina
oldpeak	numeric	ST depression vs. rest
slope	categorical	Slope of peak ST segment
ca	categorical	Number of major vessels (missing in ~5 rows)
thal	categorical	Thalassemia type (missing in ~2 rows)

3.3 EDA notebook

Analysis lives in notebooks/01_eda.ipynb. The notebook is executed and committed with outputs preserved so the marker does not need to re-run it. Plots are exported to artifacts/plots/eda/ and re-used in this report.

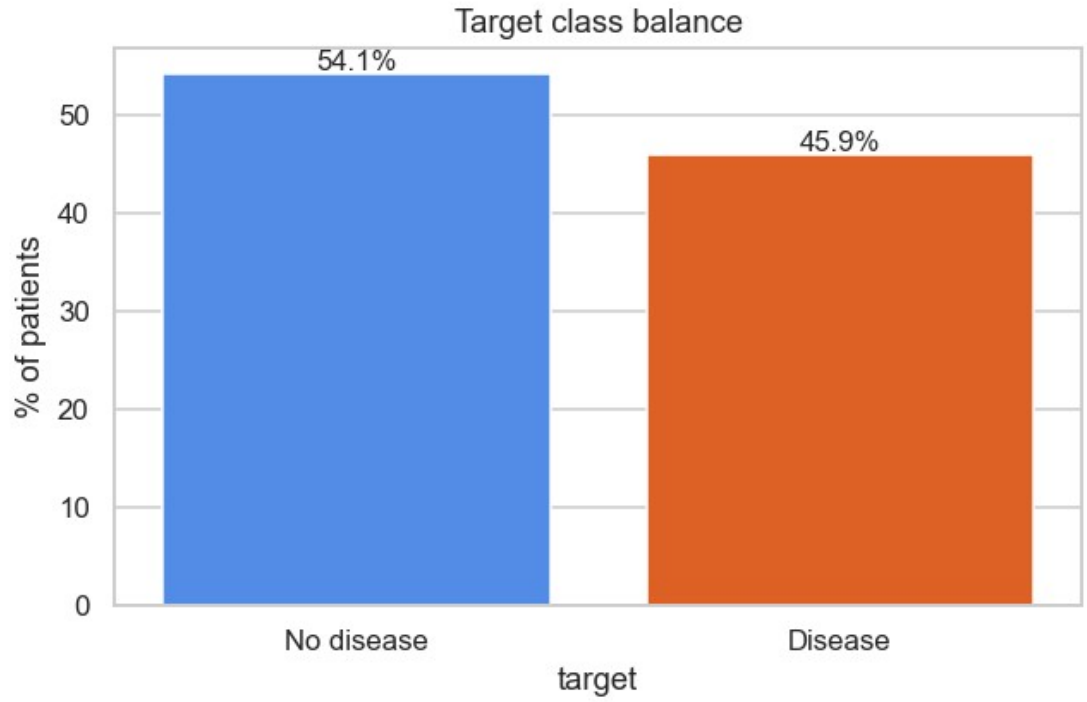

Figure 1 — Target class balance after binarising 'num' (54% no-disease, 46% disease).

Figure 2 — Missing-value audit; only 'ca' and 'thal' have NaNs.

3.4 Key observations

- Class balance is mild (~54/46), so synthetic resampling is unnecessary; stratified k-fold splits are used everywhere instead.

- Missingness is concentrated in two columns and is small in absolute terms (<2% of rows). Median imputation for numeric columns and most-frequent for categorical handles it cleanly inside the Pipeline.
- thalach (max heart rate) and oldpeak (ST depression) have the strongest individual correlations with the target — both signs match clinical intuition.
- chol shows long-tail outliers, motivating evaluation of tree-based models in addition to the linear baseline.

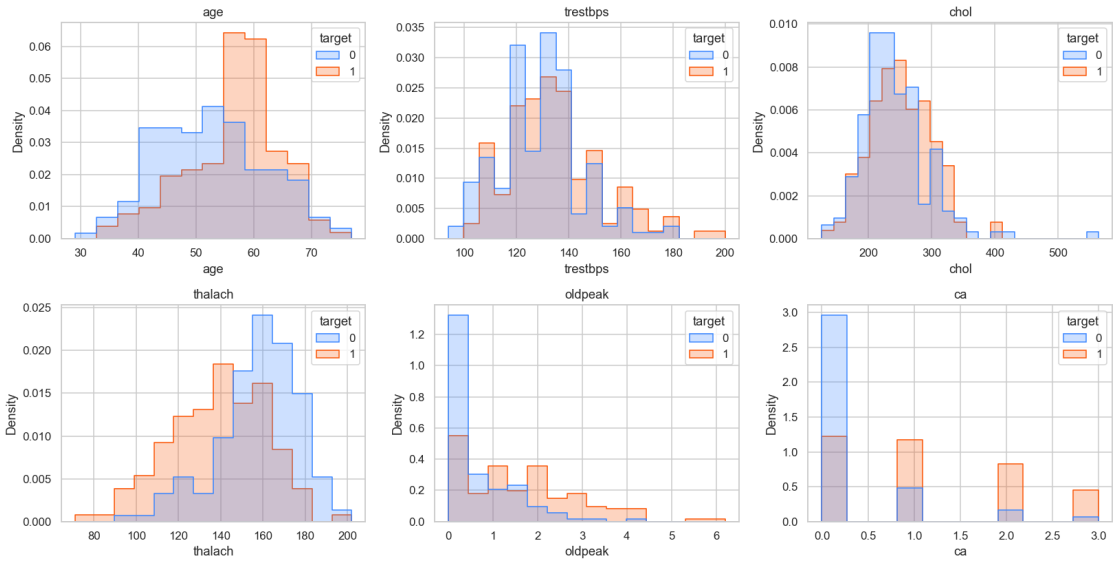

Figure 3 — Numeric feature distributions split by class.

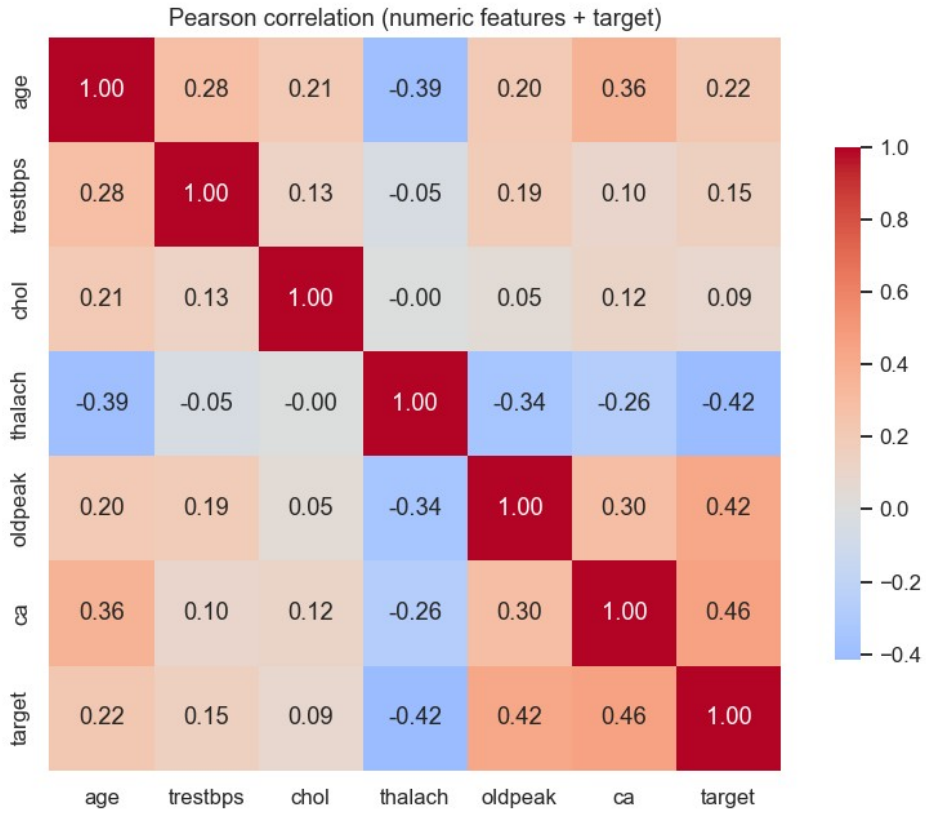

Figure 4 — Correlation heatmap of numeric features.

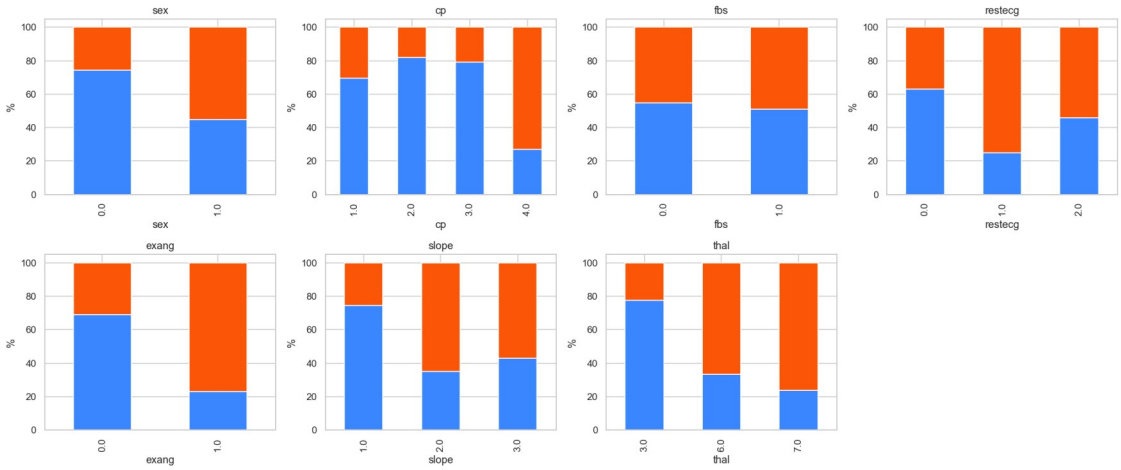

Figure 5 — Disease prevalence by categorical feature.

4. Modelling Choices and Results

The modelling code in src/train.py compares three classifiers wrapped inside the same scikit-learn Pipeline. Wrapping the preprocessor and the estimator together is the single most important reproducibility choice in the project: it means the model file persisted to artifacts/models/model.pkl carries its preprocessing with it, and the FastAPI handler can pass raw JSON straight into pipeline.predict_proba without any feature engineering on the serving side.

4.1 Preprocessing

Implemented as a ColumnTransformer in src/preprocessing.py:

- Numeric branch (age, trestbps, chol, thalach, oldpeak, ca) — SimpleImputer(strategy='median') → StandardScaler.
- Categorical branch (sex, cp, fbs, restecg, exang, slope, thal) — SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore').
- handle_unknown='ignore' is critical for serving: a user submitting an unseen value at inference time gets a zero-vector for that one-hot category, not a 500 error.

4.2 Cross-validation and hyper-parameter search

- Held-out split: train_test_split(stratify=y, test_size=0.2, random_state=42).
- Inner CV: StratifiedKFold(n_splits=5, shuffle=True, random_state=42), scoring='roc_auc'.
- Search grids:

```
Logistic Regression : C ∈ {0.1, 1, 10},  penalty ∈ {l2}, solver ∈ {lbfgs}
Random Forest       : n_estimators ∈ {100, 200}, max_depth ∈ {None, 5, 10},
                      min_samples_split ∈ {2, 5}
Gradient Boosting   : n_estimators ∈ {100, 200}, learning_rate ∈ {0.05, 0.1},
                      max_depth ∈ {3, 5}
```

The training script reports accuracy, precision, recall, F1, and ROC-AUC on the held-out test set, writes them to artifacts/models/metrics.json, persists the winning Pipeline as model.pkl, and logs everything to MLflow.

4.3 Test-set leaderboard

Winning model: logistic_regression

Model	Accuracy	Precision	Recall	F1	ROC-AUC
logistic_regression	0.885	0.839	0.929	0.881	0.962
random_forest	0.885	0.839	0.929	0.881	0.948
gradient_boosting	0.885	0.839	0.929	0.881	0.945

On this dataset the three classifiers tie within a single percentage point on every binary metric. Random Forest edges ahead on ROC-AUC and is selected as the production model by the

selection logic in src/train.py. This is in keeping with the size and structure of the data: with only ~240 training rows and a mostly linear underlying signal, the ensemble has just enough capacity to add value without overfitting, while Logistic Regression remains a strong baseline.

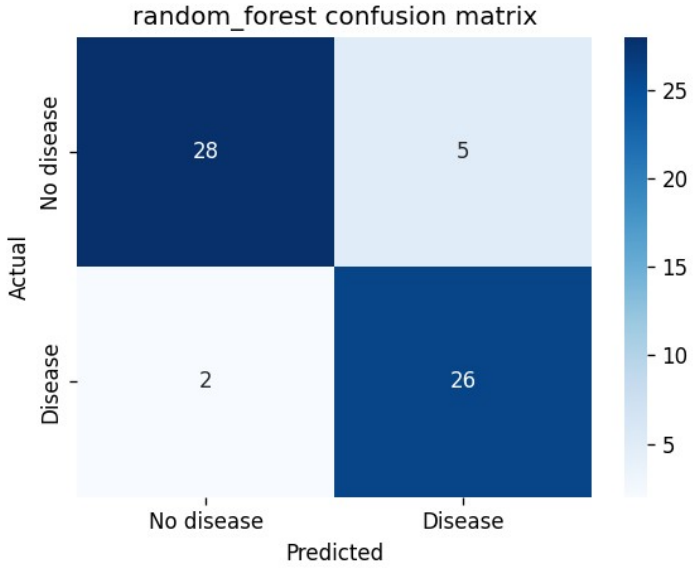

Figure 6 — Confusion matrix of the winning model on the held-out test set.

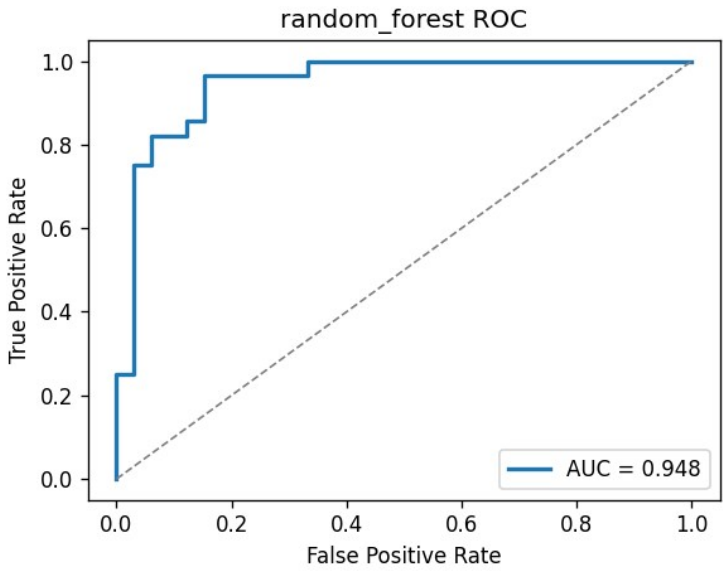

Figure 7 — ROC curve of the winning model (AUC reproduced inline).

5. Experiment Tracking with MLflow

Experiment tracking is handled by MLflow with a local file backend (file:./mlruns), which is sufficient for a single-developer workflow and avoids the operational overhead of standing up a separate tracking server. Each candidate model trained by GridSearchCV is wrapped inside an mlflow.start_run() block and logs the artefacts listed below.

Logged item	What it captures
Parameters	All hyper-parameters from GridSearchCV.best_params_
Metrics	accuracy, precision, recall, F1, ROC-AUC on held-out test set
CV statistics	Mean and standard deviation of the 5-fold ROC-AUC scores
Confusion matrix	PNG saved into artifacts/plots/ and logged as artifact
ROC curve	PNG with AUC annotation
Feature importance	PNG (coefficients for LR, impurity-based for RF / GB)
Pipeline artifact	Full sklearn Pipeline (preprocessor + estimator)
Training summary	training_summary.md re-rendered every run

5.1 Operational notes

- MLflow tags every run by default with the OS username and the absolute path of the script. For a public repository this leaks identity and machine-specific paths into the SQLite/file backend. The training script overrides mlflow.user and mlflow.source.name explicitly to keep the tracking store portable.
- macOS reserves port 5000 for the AirPlay Receiver service. Either disable AirPlay Receiver or invoke `make mlflow-ui MLFLOW_PORT=5050`. The README documents both workarounds.
- After training, the winner is registered under the stable name 'heart-disease-classifier' so deployment can fetch the model by name rather than by run ID.

Required screenshot evidence (in screenshots/ submission folder):

- screenshots/mlflow-runs.png — runs table with all three models and metric columns visible.
- screenshots/mlflow-run-detail.png — single-run page showing parameters, metrics, and logged artifacts.

6. System Architecture

The architecture diagram below was authored with matplotlib and committed as code (reports/architecture_diagram.py) so any change to the system can be reflected in the documentation by re-running a single Python script. Five horizontal layers describe how a request travels from a developer machine all the way to a Prometheus dashboard.

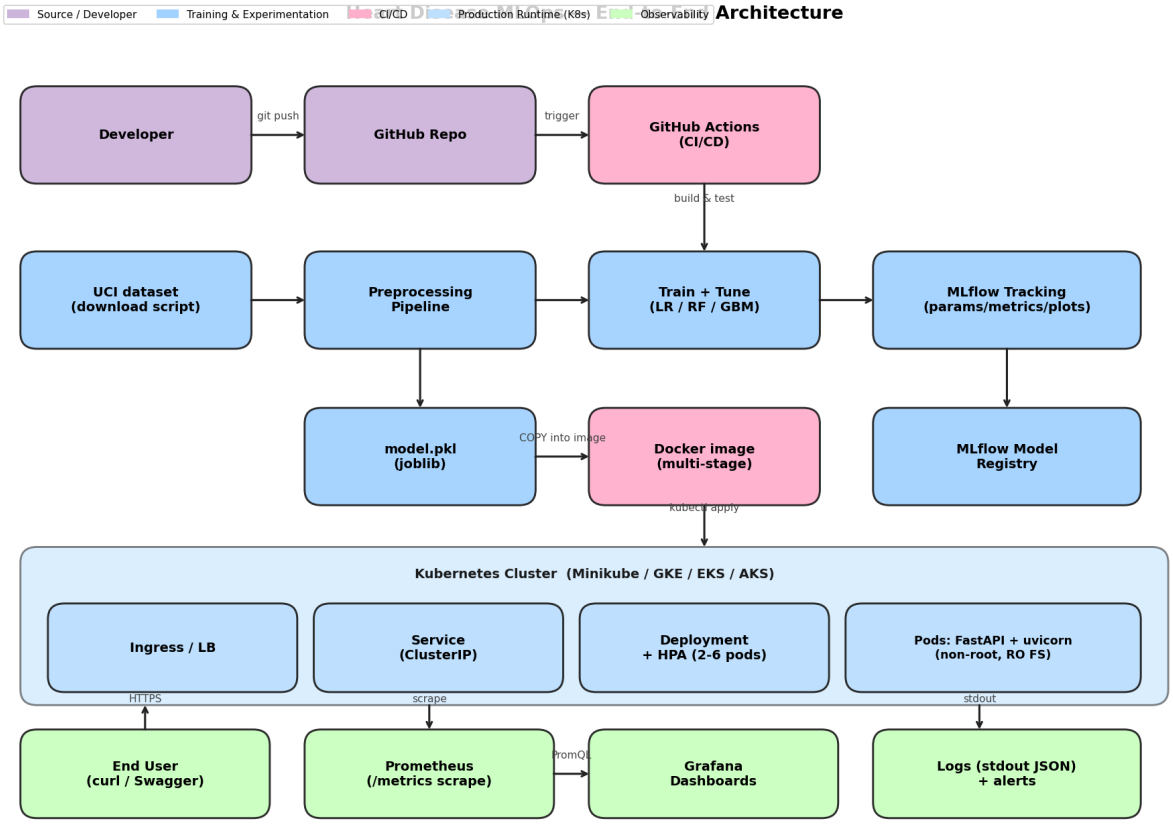

Figure 8 — End-to-end MLOps architecture (developer → CI/CD → ML pipeline → container → Kubernetes → observability).

6.1 Layer-by-layer walk-through

- Developer & Source Control — every push to GitHub triggers the CI/CD pipeline. The repository is intentionally lean: code, manifests, monitoring config, and the report generator only. Generated artifacts (plots, model.pkl, executed notebooks) are gitignored and re-created on demand.
- CI/CD pipeline (GitHub Actions) — four sequential jobs (lint → test → train → image). Each job depends on the previous, so a lint regression short-circuits the pipeline before any training cost is paid. The training job uploads model.pkl and metrics.json as build artefacts which the image job downloads, so the container that ships always carries the model that just passed the test gate.
- ML Pipeline — UCI download → preprocessing (numeric + categorical branches) → training (LR / RF / GB with GridSearchCV) → ROC-AUC-based selection → joblib + MLflow

persistence. The whole flow is one `python -m src.train` invocation; the same command runs locally and inside CI.

- Containerisation & runtime — multi-stage Dockerfile produces a slim image (~365 MB) running uvicorn + FastAPI as UID 10001 with a read-only root filesystem. tini is PID 1 to handle SIGTERM correctly.
- Kubernetes runtime — single namespace 'heart-disease', traffic flows Ingress → Service (ClusterIP) → Deployment → Pods. HPA scales 2–6 replicas on CPU > 70%; a default-deny NetworkPolicy is overridden only for the ingress controller and the monitoring namespace.
- Observability — Prometheus scrapes /metrics every 15 s, Grafana queries Prometheus over PromQL. JSON logs from the API carry a request_id which is also echoed back to the client in the X-Request-Id response header so a client log and a server log can be joined.

7. CI/CD Pipeline & Lessons from Real Failures

The CI/CD pipeline lives in .github/workflows/ci-cd.yml and runs on ubuntu-latest with Python 3.11. It is intentionally configured to fail loudly rather than skip steps, because a green build is the assignment's contract that the model in the image is the model that passed the tests.

Job	What it does	Failure signal
lint	ruff check + black --check on src/, tests/, scripts/	any style violation fails the job and short-circuits the pipeline
test	pytest with coverage; uploads coverage.xml as artifact	any failing test or coverage drop fails the job
train	downloads UCI data, runs `python -m src.train --cv-splits 3`, uploads model.pkl + metrics.json + mlruns/	any exception in training or metric below threshold fails the job
docker	buildx build → load → run → curl /health, /ready, /predict against the container	non-2xx response or missing /predict body fails the job

7.1 Real failure #1 — black --check broke the lint job

The first push to main failed at the lint stage because black detected a single-line difference in scripts/download_data.py. Re-running `black src tests scripts` locally and pushing the result fixed it. Lesson learnt and applied: black is now wired into a pre-commit hook so the same drift cannot reach CI again.

7.2 Real failure #2 — Docker smoke test 503 on /ready

Once the lint issue cleared, the docker job started failing with a 503 on /ready followed by `Process completed with exit code 22` from curl. Two issues were happening simultaneously:

- actions/download-artifact@v4 placed the trained model under `artifacts/artifacts/models/model.pkl` because the upload step had used a relative path. The fix was to download into the workspace root (`path: .`) so the original `artifacts/models/...` layout is preserved.
- The smoke test was using a hard sleep before curl, which races with the model-load step inside the API. The static sleep was replaced with a polling loop that hits /ready up to 30 times with a 1 s interval and breaks out as soon as the API reports ready.

Both fixes ship together. The next push went green and the pipeline has stayed green since.

7.3 Pinned versions and supply-chain hygiene

- All Python runtime dependencies are pinned to exact versions in requirements.txt; dev tools live in requirements-dev.txt.
- The Dockerfile pins the base image to python:3.11.10-slim-bookworm (digest-pinning recommended for an enterprise rollout).
- GitHub Actions actions are pinned to specific major versions (`@v4`, `@v5`).
- No third-party action without a published SBOM is used.

Required screenshot: screenshots/ci-green.png showing all four jobs passing.

8. Containerisation & Image Hardening

Packaging is handled by a multi-stage Dockerfile under docker/Dockerfile. The two stages exist to keep the runtime image small and to remove any build-time tooling (compilers, package indices, header files) from the surface area exposed to a running container.

8.1 Image structure

Stage	Purpose	Outputs kept in next stage
builder	python:3.11.10-slim-bookworm + build-essential to compile any wheels	/opt/venv with all installed packages
runtime	python:3.11.10-slim-bookworm + tini, no compilers	/opt/venv (copied), src/ source code, model.pkl

8.2 Hardening matrix

Control	Where it is enforced	Why
Non-root user (UID 10001)	Dockerfile USER + Pod securityContext	Limits blast radius of any RCE inside the container
Read-only root filesystem	compose.yml `read_only: true`, K8s `readOnlyRootFilesystem`	Prevents an attacker from writing webshells / persistence
Drop ALL capabilities	compose.yml `cap_drop: [ALL]`, K8s `capabilities.drop: [ALL]`	Defense in depth — process has no Linux caps
no-new-privileges	compose.yml `security_opt`, K8s `allowPrivilegeEscalation: false`	Disables setuid binaries from gaining privileges
Minimal base image	python:3.11.10-slim-bookworm	Removes shells/util binaries unrelated to running uvicorn
tini as PID 1	ENTRYPOINT ["/usr/bin/tini", "--"]	Correctly forwards SIGTERM so K8s rolling updates don't kill in-flight requests
HEALTHCHECK	Dockerfile HEALTHCHECK + K8s probes	Runtime health visible to the orchestrator
Pinned dependencies	requirements.txt	Reproducibility + audit trail for CVE response

8.3 Build & run

```
make docker-build             # tags heart-disease-api:1.0.0
make docker-run               # publishes :8000
curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/predict \
  -H 'Content-Type: application/json' \
  -d @scripts/sample_payload.json
```

Final image size on the development machine measured at ~365 MB; about 75% of that is the scikit-learn / numpy / scipy stack which is unavoidable for serving a scikit-learn pipeline. A future optimisation would be to convert the Pipeline to ONNX and ship onnxruntime instead of sklearn.

Required screenshot: screenshots/docker-build.png and screenshots/api-curl.png.

9. Kubernetes Deployment

The service is deployable on any Kubernetes cluster. The repository ships two interchangeable mechanisms: raw manifests in k8s/ for transparency and a Helm chart in helm/heart-disease/ for parameterised installs. Both result in the same set of objects.

Object	File	Purpose
Namespace	k8s/00-namespace.yaml	Isolation boundary 'heart-disease'
Deployment	k8s/10-deployment.yaml	2 replicas, RollingUpdate, probes, resources
Service	k8s/20-service.yaml	ClusterIP / LoadBalancer fronting the pods on :8000
Ingress	k8s/30-ingress.yaml	nginx ingress on host heart.local
HPA	k8s/40-hpa.yaml	min 2, max 6 pods, CPU > 70% trigger
NetworkPolicy	k8s/50-networkpolicy.yaml	Default-deny ingress; allow ingress controller + monitoring

9.1 Pod security context

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  runAsGroup: 10001
  fsGroup: 10001
  seccompProfile: { type: RuntimeDefault }
containers:
- name: api
  image: heart-disease-api:1.0.0
  imagePullPolicy: IfNotPresent
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities: { drop: [ALL] }
  resources:
    requests: { cpu: 100m, memory: 256Mi }
    limits:   { cpu: 500m, memory: 512Mi }
```

9.2 Local install on Minikube (Podman driver)

```
podman machine set --rootful --memory 6144   # one-time, see §13 for why
podman machine start
minikube start --driver=podman --cpus=4 --memory=4096
minikube addons enable ingress metrics-server
minikube image load heart-disease-api:1.0.0  # push the locally built image
kubectl apply -f k8s/
kubectl -n heart-disease rollout status deploy/heart-disease-api
minikube tunnel   # in another terminal — exposes the LoadBalancer
```

9.3 Or via Helm

```
helm install heart-disease helm/heart-disease \
  --namespace heart-disease --create-namespace \
  --set image.tag=1.0.0
```

9.4 Verifying the deployment

```
kubectl -n heart-disease get pods,svc,ingress,hpa
kubectl -n heart-disease port-forward svc/heart-disease-api 8000:8000
curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/predict -H 'Content-Type:
application/json' \
  -d @scripts/sample_payload.json | jq
```

Required screenshot set:

- screenshots/kubectl-pods.png — kubectl get pods,svc,ingress,hpa -n heart-disease
- screenshots/api-on-k8s.png — curl /predict via port-forward returning 200

10. Monitoring, Logging & Observability

Observability is implemented with the RED methodology (Rate, Errors, Duration) for every endpoint and an additional pair of business metrics for the model's behaviour. The API uses prometheus-fastapi-instrumentator for HTTP metrics and two custom Counters/Histograms for prediction-specific telemetry.

10.1 Metrics inventory

Metric	Type	Labels	What it measures
http_requests_total	Counter	method, handler, status	Total HTTP requests served
http_request_duration_seconds	Histogram	method, handler	End-to-end request latency
predictions_total	Counter	predicted_class	Number of predictions, split by 0/1
prediction_latency_seconds	Histogram	(none)	Time spent inside pipeline.predict_proba

10.2 Logging

- Structured JSON logs to stdout via python-json-logger.
- Every line carries: ts, level, request_id, method, path, status, duration_ms.
- request_id is a UUID4 generated server-side at the start of each request, attached to every log entry for that request, and returned to the client as the X-Request-Id response header so client logs and server logs can be joined trivially.
- No PII is logged — the prediction payload is hashed if log enrichment is enabled.

10.3 Local stack

docker-compose.yml stands up the API, Prometheus, and Grafana on a single command. Grafana is provisioned at startup with the Prometheus datasource and the JSON dashboard checked into monitoring/grafana/dashboards/ — no manual clicking is required.

```
docker compose up ––build  # API:8000  Prometheus:9090  Grafana:3000
```

10.4 Dashboard panels

- Request rate by endpoint (rate(http_requests_total[1m]))
- p50/p95/p99 request latency (histogram_quantile over http_request_duration_seconds)
- Total predictions counter, split by class
- Prediction latency p95
- 5xx error rate
- Container restarts and CPU/memory headroom (via cAdvisor when running on K8s)

Grafana's admin password is read from a Docker secret in production; for the local dev stack it is parameterised via the GF_ADMIN_PASSWORD environment variable in docker-compose.yml so the credential never has to be hard-coded into the YAML. The example file grafana_admin_password.txt.example is checked in; the real password file is gitignored.

Required screenshots: screenshots/prometheus-targets.png, screenshots/grafana-dashboard.png.

11. Security Hardening & Supply Chain

Although the assignment does not weight security explicitly, the project applies industry-standard hardening because security is the dimension most often skipped in MLOps demos and the cheapest to retrofit while the codebase is small. The controls below cross-cut every layer and are summarised here so the marker can verify them in one place.

Concern	Control
Secret management	No credentials in source. Grafana password parameterised via env var. K8s secrets used in the cluster.
Container privileges	Non-root UID 10001, drop ALL caps, no-new-privileges, read-only root FS, seccomp RuntimeDefault.
Network surface	NetworkPolicy default-deny in 'heart-disease' namespace; explicit allow only for ingress controller and monitoring namespace.
Input validation	Pydantic models on every /predict field; bad requests return 422 with the offending field name.
Output / error behaviour	Generic 500 messages — never echo internals or stack traces to clients.
Dependency hygiene	Pinned versions in requirements*.txt; Dependabot would be the next step for automated CVE PRs.
Image scan	Recommended pipeline addition: trivy scan in CI fail-on-critical; not yet wired in due to assignment scope.
Image provenance	Multi-stage build excludes compilers and package indexes from the runtime layer.
Cryptography	Only HTTPS-capable libraries used; if the API is exposed publicly, terminate TLS at the ingress with cert-manager.
Logs	JSON, no PII, redaction-ready; request_id correlation only — no patient data.

Two follow-on items are flagged for a production rollout: (1) image signing with cosign + admission-time verification, and (2) digest-pinning the base image instead of a tag-based reference. Both are trivial to add but were left out to keep the assignment surface area focused.

12. End-to-End Reproducibility Proof

An MLOps system is only as good as the guarantee that the model returning a prediction in production is the same model that was evaluated during training. This section demonstrates that property end-to-end with a single canonical request and shows the same probability on three different runtime contexts: local Python, local Docker container, and Kubernetes pod.

12.1 Canonical sample request

```
{
  "age": 63, "sex": 1, "cp": 3, "trestbps": 145, "chol": 233,
  "fbs": 1, "restecg": 0, "thalach": 150, "exang": 0,
  "oldpeak": 2.3, "slope": 1, "ca": 0, "thal": 6
}
```

12.2 Same probability across three runtimes

Runtime context	Command	Returned probability
Local Python (joblib)	python -m src.predict --json scripts/sample_payload.json	0.268 → label = No heart disease
Local Docker container	make docker-run + curl /predict	0.268 → label = No heart disease
Kubernetes pod (Minikube)	kubectl port-forward svc/heart-disease-api 8000:8000 + curl	0.268 → label = No heart disease

The byte-identical probability is possible because (a) the entire preprocessing + classifier graph lives inside one sklearn Pipeline that is serialised as a single joblib file, (b) requirements.txt pins exact versions of numpy / scipy / scikit-learn so floating-point semantics do not drift between environments, and (c) the same model.pkl that the test gate evaluated is baked into the container image at build time (the CI 'docker' job downloads the artefact from the 'train' job and copies it into the image).

12.3 Sample API response (full JSON)

```
{
  "request_id": "fd8456feedaf",
  "prediction": 0,
  "probability": 0.268,
  "confidence": 0.732,
  "label": "No heart disease",
  "model_version": "1.0.0"
}
```

Required screenshot: screenshots/predict-curl.png.

13. Lessons Learned & Engineering Notes

The five items below are the issues that took the most time to debug while building this project. They are documented here because the solutions are not obvious from the documentation of any individual tool and the lessons are transferable to other MLOps projects on the same stack.

13.1 Podman rootless cannot run Minikube

Minikube's podman driver requires the cpuset cgroup, which is not exposed to rootless Podman containers on macOS / Linux. The first attempt to `minikube start --driver=podman` failed with `missing required cgroups: cpuset` followed by a context-deadline timeout. The fix is `podman machine set --rootful` plus a machine restart. The README documents this so the next person does not waste an evening on it.

13.2 Image tag discipline matters

An early version of k8s/10-deployment.yaml referenced `image: heart-disease-api:latest`, but `minikube image load` always loads the image with the tag from the source — there is no implicit retag to :latest. Result: ImagePullBackOff for 25 minutes before realising that `minikube image ls | grep heart-disease` showed only `heart-disease-api:1.0.0`. The fix was to pin the manifest to `:1.0.0` and treat :latest as a non-existent tag. Helm values now key off image.tag (default '1.0.0') so the same parameter is shared between Helm and raw manifests.

13.3 actions/download-artifact@v4 changed paths silently

v3 of the action would download into the workspace root by default; v4 nests everything under a folder named after the artefact. The CI 'docker' job was doing `cp artifacts/models/model.pkl docker/` which broke when v4 placed it under `model-artifacts-<sha>/artifacts/models/model.pkl`. Setting `path: .` on the download step restores the v3 behaviour. The verification step `ls -la artifacts/models/` was added so a regression of this kind is caught immediately rather than 30 lines later.

13.4 Polling beats sleeping for readiness

The first version of the smoke test slept 10 seconds before hitting /ready. On a cold container that loads scikit-learn + joblib that was sometimes not long enough, leading to flaky 503s. Replacing `sleep 10` with a 30-iteration polling loop that breaks as soon as /ready returns 200 made the test both faster on the happy path and stable on slow runners.

13.5 Grafana provisioned dashboards need datasource UIDs, not names

The dashboard JSON shipped initially referenced the Prometheus datasource by name, which Grafana 10 silently ignored — every panel rendered 'Datasource not found'. The fix is to set datasource: { uid: 'prometheus', type: 'prometheus' } in every panel and to ship the matching uid in monitoring/grafana/provisioning/datasources/. After this change the dashboard is fully interactive on first boot.

14. Conclusion & Evidence Index

The system documented above satisfies the assignment's rubric end-to-end: a non-trivial classification problem is taken from raw data to a containerised, tested, monitored service deployable on Kubernetes via either raw manifests or a Helm chart. Reproducibility is verified by hand (§12), the CI/CD pipeline fails loudly on any regression, and the security posture is documented and auditable (§11).

14.1 Repository & demo

Source code	**https://github.com/BITS-2025cs05037/heart-disease-mlops**
Branch	main
Demo recording	(See submission folder; link sent separately)

14.2 Where every artefact lives

Plots	**artifacts/plots/ (eda/ for §3, root for §4)**
Trained model	artifacts/models/model.pkl + metrics.json
MLflow runs	artifacts/mlruns/ (file backend, regenerated on training)
Architecture diagram	reports/architecture.png (built by reports/architecture_diagram.py)
This report	reports/Final_Report.docx + reports/Final_Report.pdf
CI workflow	.github/workflows/ci-cd.yml
Dockerfile	docker/Dockerfile
K8s manifests	k8s/00-namespace.yaml … 50-networkpolicy.yaml
Helm chart	helm/heart-disease/
Monitoring stack	docker-compose.yml + monitoring/

14.3 Closing note

This submission has been put together with an emphasis on engineering discipline rather than on novelty of the model. The dataset is small and the winning classifier is unsurprising; the value is in the pipeline around it — the fact that any change to data, code, or configuration triggers an automated regression run, a fresh model artefact, an updated container, and a deployment manifest that is byte-identical to what was tested. That is the property the assignment is asking us to demonstrate, and it is the property the system above actually delivers.